

Intelligence artificielle · Sécurité applicative

Injection de prompt : la faille que personne ne sait refermer

Notre plateforme embarque un assistant conversationnel qui accompagne les joueurs. C'est un service rendu, et c'est aussi une surface d'attaque dont la vulnérabilité principale n'a, à ce jour, aucun correctif complet.

Date de veille 10 juillet 2026

Rédacteur Sami El Abdallaoui, stagiaire veille et contenus

Déclencheur Présence d'un assistant IA dans le parcours joueur

Public visé Équipe technique OpenCyberAI

Temps de lecture 6 minutes

À retenir

- L'injection de prompt occupe la première place du Top 10 OWASP pour les applications à base de modèles de langage. Elle y est depuis la première édition et n'en est jamais sortie.
- La cause est **architecturale** : instructions et données partagent le même canal, la fenêtre de contexte. Il n'existe pas d'équivalent de la requête préparée qui séparerait les deux, comme on le fait contre l'injection SQL.
- Conséquence directe : toutes les défenses connues abaissent la probabilité de succès, aucune ne la ramène à zéro.
- L'injection **indirecte** est la plus insidieuse : la charge n'est pas tapée par l'utilisateur, elle est déposée dans un contenu que le modèle va lire — une page, un document, un champ de données.
- La bonne posture est celle du moindre privilège : limiter ce que l'assistant a le droit de faire et de voir, plutôt que d'espérer filtrer parfaitement ce qu'on lui dit.

Pourquoi la comparaison avec l'injection SQL s'arrête à mi-chemin

L'analogie est tentante et elle est juste sur le principe : dans les deux cas, une donnée fournie par l'utilisateur finit interprétée comme une instruction. Mais l'injection SQL a une solution propre et définitive — la requête préparée, qui sépare structurellement le code des paramètres. La base ne peut plus confondre les deux, quel que soit le contenu envoyé.

Rien de tel n'existe pour un modèle de langage. La consigne système, l'historique de conversation et les données à traiter arrivent dans le même flux de texte, et c'est le modèle lui-même qui décide de ce qui compte comme une instruction. On peut l'entraîner à mieux résister, ajouter des filtres, encadrer le format des entrées — on ne peut pas lui garantir la séparation.

C'est ce qui distingue cette vulnérabilité de presque toutes les autres : elle ne se corrige pas, elle se contient.

Directe et indirecte

Injection directe

L'utilisateur formule lui-même une consigne destinée à détourner le comportement de l'assistant : lui faire ignorer ses règles, lui faire révéler sa consigne système, lui faire produire ce qu'il n'est pas censé produire.

Injection indirecte

Le modèle lit un contenu externe — page web, fichier, champ d'une base — dans lequel un tiers a glissé des instructions. L'utilisateur n'a rien demandé de malveillant, et peut même ne rien voir passer. Des exploitations sans aucune interaction de la victime ont été documentées en production dès 2025, notamment sur des assistants intégrés à des suites bureautiques.

Le vecteur se déplace ensuite vers les agents : un assistant qui peut appeler des outils, lire des fichiers ou déclencher des actions transforme une réponse détournée en action détournée. La question n'est plus ce que le modèle dit, mais ce qu'il fait.

Ce qui limite réellement les dégâts

- **Moindre privilège sur les outils** — ne donner à l'assistant que les capacités strictement nécessaires. Un assistant qui ne peut pas écrire ne peut pas être détourné pour écrire.
- **Validation humaine** sur toute action à conséquence : rien de sensible ne s'exécute sans confirmation explicite.
- **Cloisonnement du contexte** — ne jamais placer dans la fenêtre de contexte une information que l'assistant ne doit pas divulguer. S'il l'a, il peut la sortir.
- **Filtrage en entrée et en sortie**, utile mais partiel : c'est une couche, pas une garantie.
- **Tests adverses réguliers** — traiter l'assistant comme une surface à attaquer, avec une campagne dédiée, et pas seulement comme une fonctionnalité à vérifier.

Ce que cela change pour notre plateforme

— Notre assistant accompagne un joueur pendant qu'il résout un exercice. S'il a la correction en contexte, un élève un peu curieux la lui fera dire — ce n'est pas une faille de sécurité au sens classique, mais c'est la fin de l'évaluation.

— Recommandation prioritaire : ne jamais charger la solution attendue dans le contexte de l'assistant. La contrainte pédagogique et la contrainte de sécurité disent ici la même chose.

— Prévoir une passe de test adverse sur l'assistant, au même titre qu'une campagne de test fonctionnel : tenter de lui faire divulguer sa consigne, la réponse, ou de le faire sortir de son rôle.

— Sujet pédagogique à fort potentiel : nos élèves utilisent des assistants tous les jours. Un scénario d'injection indirecte leur montrerait qu'un outil peut être manipulé par un contenu qu'ils n'ont pas écrit.

Sources consultées

- OWASP GenAI Security Project — LLM01, Prompt Injection, Top 10 for LLM Applications
- OWASP — Top 10 for Agentic Applications, 2026
- MITRE / CVE-2025-32711 — injection de commande via IA dans Microsoft 365 Copilot
- Travaux publiés sur EchoLeak, première exploitation zéro-clic documentée en production, 2025
- Cisco — « Prompt injection is the new SQL injection », mars 2026
- NIST AI 600-1 — profil IA générative du cadre de gestion des risques

Réserve : le sujet évolue vite et une partie de la littérature disponible est produite par des éditeurs vendant des solutions de filtrage. Les éléments retenus ici sont ceux qui font consensus dans les publications OWASP et dans la littérature académique, et non les promesses de remédiation totale.